

# Atividade 2 - Automação Web Cookie Clicker (Selenium ou Playwright)

## Contexto

Você irá implementar um **sistema de automação web** capaz de jogar **Cookie Clicker** automaticamente, utilizando **Selenium** ou **Playwright**.

A solução deve ser construída com **arquitetura robusta e consistente**, seguindo **princípios SOLID** e aplicando **padrões de design** de forma padronizada em todo o projeto.

---

## Objetivo

Criar um bot que:

1. Acesse o site do Cookie Clicker
  2. Clique no cookie de forma **altamente otimizada**
  3. Compre upgrades e melhorias automaticamente
  4. Escolha **sempre a melhor melhoria disponível** com base em uma estratégia dinâmica
- 

## Requisitos Funcionais Obrigatórios

### 1) Acessar o jogo

- Abrir o site do **Cookie Clicker (versão web)**
- Garantir que o jogo esteja pronto antes de iniciar (aguardar carregamento de elementos essenciais)

### 2) Otimização de cliques

- Clicar repetidamente no cookie da forma **mais otimizada possível**
- Evitar desperdícios de performance (ex.: sleeps longos, loops ineficientes, re-buscas excessivas no DOM)

É permitido (e incentivado) usar **concorrência** (ex.: multithreading) para maximizar performance, desde que sua solução permaneça consistente e fácil de manter.

### 3) Compra automática de melhorias

O bot deve:

- Detectar quando upgrades/melhorias estiverem disponíveis para compra
  - Comprar upgrades automaticamente
  - **Comprar sempre a melhor melhoria disponível** no momento da compra
- 

## Desafio de Algoritmo (Obrigatório)

O diferencial desta atividade é a decisão inteligente de compras, baseada em leitura dinâmica do estado do jogo.

### 1) Reconhecimento dinâmico de upgrades

- O algoritmo deve identificar **quais melhorias estão disponíveis** em tempo real
- A solução **não pode** depender de uma lista fixa (hardcoded) de nomes/ordens de compra
- A lógica deve ser baseada em dados obtidos do DOM (ex.: custo, efeito, ganho estimado)

### 2) Critério de compra inteligente (não pode ser “sempre o mais barato”)

- O algoritmo deve evitar o comportamento trivial de comprar sempre a melhoria mais barata
- Deve existir um critério justificável e documentado, por exemplo:
  - **custo-benefício** (ganho por custo)
  - **tempo de retorno** (payback)
  - **impacto em produção/eficiência**
  - estratégia híbrida (early game vs mid game)

No README, explique claramente o critério e por que ele leva a escolhas melhores do que comprar “o mais barato”.

### 3) Otimização do fluxo (clique x compra)

- Minimizar pausas e gargalos entre “clique” e “analisar/comprar”
  - Controlar o ciclo do bot para manter alta taxa de cliques sem perder oportunidades de compra
- 

## Requisitos de Arquitetura e SOLID (Obrigatório)

Todas as implementações devem:

- Aplicar **SOLID** de forma verificável

## Padrões recomendados (você deve justificar no README)

Use uma arquitetura consistente, por exemplo:

- **Hexagonal**

E aplique padrões quando fizer sentido, como:

- **Repository** (persistência — ex.: logs, métricas, histórico)
  - **Service / UseCase** (casos de uso — ex.: “executar clique”, “avaliar upgrades”, “comprar melhor upgrade”)
  - **Factory** (criação de objetos complexos — ex.: driver, browser context, config)
  - **Providers** (ex.: provider de tempo, provider de estado do jogo, provider de seleção)
  - **DTOs** (contratos de entrada/saída — ex.: UpgradeDto, GameStateDto)
  - **Controllers** (orquestração — ex.: iniciar bot, parar bot, executar ciclo)
  - **Routes** (se você expor interface extra, ex.: API local opcional)
  - **Request DTOs** (se aplicável, para entradas externas)
- 

## Requisitos Técnicos

- Implementar com **Selenium** ou **Playwright**
  - Implementar logs claros contendo, no mínimo:
    - taxa de cliques (ex.: cliques/segundo por intervalo)
    - upgrades detectados e seus custos
    - decisões de compra com justificativa (por que escolheu aquele upgrade)
    - compras realizadas e saldo restante
  - Tratar erros comuns:
    - elemento não encontrado / seletor mudou
    - carregamento lento
    - falhas intermitentes do navegador
- 

## Restrições

- Não é permitido resolver “na força bruta” usando sleeps fixos e longos que diminuam performance.

- Não é permitido hardcode de ranking fixo de upgrades (ex.: “sempre comprar X primeiro”).
  - O algoritmo deve ser **dinâmico**, reagindo ao estado atual do jogo.
- 

## Entrega Obrigatória (1 repositório GitHub)

Você deve entregar **um repositório** contendo:

- Código fonte completo
  - Gerenciamento de dependências ( `requirements.txt` ou `pyproject.toml` )
  - Instruções de execução (incluindo dependências do browser/driver)
  - README com:
    - explicação da arquitetura (incluindo referência a SOLID e Hexagonal)
    - estratégia de compra e critério de decisão
    - exemplos de logs e execução
- 

## Critérios de Avaliação

Você será avaliado por:

1. Cumprimento dos requisitos funcionais
  2. Eficiência do sistema (taxa de cliques e estabilidade)
  3. Qualidade da estratégia de compra (dinâmica e justificável)
  4. Arquitetura consistente (Hexagonal + padrões aplicados com sentido)
  5. Aplicação verificável de SOLID
  6. Tratamento de erros e clareza dos logs
  7. README completo e bem explicado
- 

## Bônus (Opcional)

- Implementar múltiplas estratégias e selecionar via configuração
- Comparar estratégias e gerar um relatório simples (ex.: compras por minuto)
- Modo headless + modo debug visual

- Concorrência segura (ex.: thread dedicada para compra e thread dedicada para clique, com coordenação)